

HCI Models, Theories, and Frameworks

Edited by John M. Carroll

MacKenzie, S. "Chapter 3: Motor Behavior Models for Human-Computer Interaction." *HCI Models, Theories, and Frameworks: toward a Multidisciplinary Science*. Ed. John M. Carroll. San Francisco, CA: Morgan Kaufmann, 2003. 27-54.

3

CHAPTER

Motor Behavior Models for Human-Computer Interaction

I. Scott MacKenzie York University, Toronto, Canada

3.1

MOTIVATION

The movement of body and limbs is inescapable in human-computer interaction (HCI). Whether browsing the Web or intensively entering and editing text in a document, our arms, wrists, and fingers are at work on the keyboard, mouse, and desktop. Our head, neck, and eyes move about, attending to feedback that marks our progress. This chapter is motivated by the need to match the movement limits, capabilities, and potential of humans with input devices and interaction techniques on computing systems. Our focus is on models of human movement relevant to human-computer interaction. Some of the models discussed here emerged from basic research in experimental psychology, whereas others emerged from, and were motivated by, the specific need in HCI to model the interaction between users and physical devices, such as mice and keyboards.

As much as we focus on specific models of human movement and user interaction with devices, this chapter is also about models in general. We will say a lot about the nature of models, what they are, and why they are important tools for the research and development of human-computer interfaces.

3.2

OVERVIEW: MODELS AND MODELING

By its very nature, a model is a simplification of reality. However, a model is useful only if it helps in designing, evaluating, or otherwise providing a basis for understanding the behavior of a complex artifact such as a computer system. It is convenient to think of models as lying in a continuum, with analogy and

metaphor at one end and mathematical equations at the other. Most models lie somewhere inbetween. Toward the metaphoric end are *descriptive models*; toward the mathematical end are *predictive models*. These two categories are our particular focus in this chapter, and we shall visit a few examples of each. Two models will be presented in detail and in case studies: Fitts' (1954, 1964) model of the information-processing capability of the human motor system, and Guiard's (1987) model of bimanual control.

Fitts' model is a mathematical expression emerging from the rigors of probability theory. It is a predictive model at the mathematical end of the continuum, to be sure, yet when applied as a model of human movement it has characteristics of a metaphor. Guiard's model emerged from a detailed analysis of how humans use their hands in everyday tasks, such as writing, drawing, playing a sport, or manipulating objects. It is a descriptive model, lacking in mathematical rigor but rich in expressive power.

Neither Guiard's model nor Fitts' model was motivated by issues in human-computer interaction. Yet, today, both are commonly used in the research and development of interactive systems. The reason is simple: They are useful! They are useful because they provide a simplification of the complex interactions between humans and computers. This simplification allows designers to understand and anticipate the impact of a design in a meaningful context. A founding principle of the field of human-computer interaction is that it is multidisciplinary. The field combines work in other disciplines, most notably psychology, cognitive science, and sociology. Fitts' and Guiard's models emerged from basic research in an area within experimental psychology known as *psychomotor behavior* or, simply, *motor control*. Both are widely applicable to many disciplines. The field of human-computer interaction is a beneficiary of this basic research.

Before discussing Fitts' and Guiard's work in detail, we begin with a general discussion on predictive and descriptive models, and we present a few other relevant models within these categories.

3.2.1 Predictive Models

Predictive models, sometimes called *engineering models* or *performance models* (Card, Moran, & Newell, 1983, p. 411; Marchionini & Sibert, 1992), are widely used in many disciplines. In human-computer interaction, predictive models allow metrics of human performance to be determined analytically without undertaking time-consuming and resource-intensive experiments. Predictions so generated are *a priori*: They allow a design scenario to be explored hypothetically,

without implementing a real system and gathering the same performance metrics through direct observation on real users. The benefits are obvious.

Hick-Hyman Law

There are many examples of predictive models, some specific to human-computer interaction, others guided by basic theory in perception or motor control. We have already noted that both Fitts' and Guiard's models emerged from basic research in motor control, and we shall have more to say about these shortly. Another example is the Hick-Hyman law for choice reaction time (Hick, 1952; Hyman, 1953). This law takes the form of a prediction equation. Given a set of n stimuli, associated one for one with n responses, the time to react (RT) to the onset of a stimulus and to make the appropriate response is given by

$$RT = a + b \log_2(n) \quad (3.1)$$

where a and b are empirically determined constants. The Hick-Hyman law has surfaced in a few contexts in interactive systems. Card et al. (1983, p. 74) provide an example of a telephone operator selecting among ten buttons when the light behind a button comes on. Landauer and Nachbar (1985) applied the Hick-Hyman law in measuring and predicting the time to select items in hierarchical menus. Besides confirming the suitability of the law to this category of task, they also found empirical support for their conclusion that breadth should be favored over depth in hierarchical menus. More recently, we have found the Hick-Hyman law useful in predicting text-entry rates on soft keyboards with non-Qwerty layouts (MacKenzie, Zhang, & Soukoreff, 1999; Soukoreff & MacKenzie, 1995). For non-Qwerty layouts, users must visually scan the keyboard to find the desired letter. The act of finding the desired letter among a set of randomly positioned letters is appropriately modeled by the relationship in Equation 3.1.

Keystroke-Level Model

There are numerous predictive models developed specifically for exploring design scenarios for human-computer interfaces. One of the earliest and certainly one of the most comprehensive is the *keystroke-level model* (KLM) by Card et al. (1980; 1983, chap. 8). The KLM was developed as a practical design tool, the goal being to predict the time to accomplish a task on a computer system. The model predicts expert error-free task-completion times, given the following input parameters:

- ♦ a task or series of subtasks
- ♦ method used
- ♦ command language of the system
- ♦ motor-skill parameters of the user
- ♦ response-time parameters of the system

A KLM prediction is the sum of the subtask times and the required overhead. The model includes four motor-control operators (K = key stroking, P = pointing, H = homing, D = drawing), one mental operator (M), and one system-response operator (R):

$$T_{\text{EXECUTE}} = t_k + t_p + t_h + t_d + t_m + t_r \quad (3.2)$$

Some of the operations are omitted or repeated, depending on the task. For example, if a task requires n keystrokes, t_k becomes $n \times t_k$. Each t_k operation is assigned a value according to the skill of the user, with values ranging from $t_k = 0.08$ s for highly skilled typists to $t_k = 1.20$ s for a typist working with an unfamiliar keyboard. The pointing operator, t_p , is based on Fitts' law. More on this later.

Since its introduction, the KLM has surfaced in many contexts in HCI, such as map digitizing (Haunold & Kuhn, 1994), predicting usage patterns in editors (Toleman & Welsh, 1996), generating power key assignments to reduce typing keystrokes (Nicols & Ritter, 1995), predicting user performance with hierarchical menus (Lane, Napier, Batsell, & Naman, 1993), or predicting text-entry performance for physically challenged users in a word-prediction system (Koester & Levine, 1994).

3.2.2 Descriptive Models

Descriptive models are of a vastly different genre than predictive models. Although they generally do not yield an empirical or quantitative measure of user performance, their utility is by no means second to predictive models. Simply put, descriptive models provide a framework or context for thinking about or describing a problem or situation. Often the framework is little more than a verbal or graphic articulation of categories or identifiable features in an interface. Nevertheless, the simple possession of such a framework arms the designer with a tool for studying and thinking about the user-interaction experience. A simple example is developed in the next section.

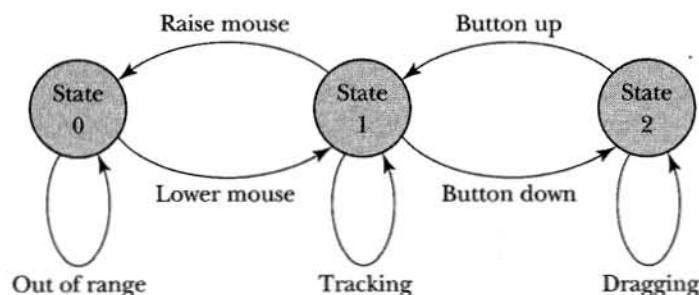
Key-Action Model

Computer keyboards today contain a vast array of buttons or keys. Most systems use a variation of the 101-style keyboard, with a row of function keys across the top and a numeric keypad on the right. Have you ever thought about the operation of keys on a keyboard? Here's a descriptive model for this purpose. Let's call it the *key-action model* (KAM). In our KAM, keyboard keys are categorized as either *symbol keys*, *executive keys*, or *modifier keys*. Symbol keys deliver graphic symbols—typically, letters, numbers, or punctuation symbols—to an application such as a text editor. Executive keys invoke actions in the application or at the system level or meta level. Examples include ENTER, F1, or ESC. Modifier keys do not generate symbols or invoke actions, but, rather, set up a condition necessary to modify the effect of a subsequently pressed key. Examples include SHIFT or ALT. And that's about it for the KAM. It's a simple model, to be sure. It has a name, and it identifies three categories of keys, providing for each a name, a definition, and examples. What do you think of this model? Is it correct? Is it flawed? Do all keyboard keys fit the model? Can you think of additional categories or subcategories to improve the model or to make it more accurate or more comprehensive? Do some keys have features of more than one category? Can you think of a graphical illustration of the model to improve its expressive power? Is the model useful?

The substance of these questions is evidence of the power of descriptive models such as the KAM. The model piques our interest and suggests aspects of keyboard operation that merit consideration, particularly if a new design is contemplated. The most important question is the last. There is no greater measure of the merit of a model than its ability to tease out critical arguments on the potential, the capabilities, and the limitations in a particular interaction domain. We will revisit the KAM later in a case study on the affordances of the desktop interface for two-handed interaction.

Three-State Model of Graphical Input

Another example is Buxton's *3-state model* for graphical input devices (Buxton, 1990). The model is a simple expression of the operation of computer pointing devices in terms of state transitions. It is described simply as a vocabulary to recognize and explore the relationship between pointing devices and the interaction techniques they afford (Buxton, 1990, p. 449). In this sense, it is a paradigm of descriptive modeling. The three states are identified in Figure 3.1, annotated for mouse interaction.



FIGURE

3.1

Buxton's 3-state model of graphical input (after Buxton, 1990).

Left to right in Figure 3.1, the states are *Out of range* (State 0) for clutching or repositioning a mouse on a mouse pad, *Tracking* (State 1) for moving a tracking symbol such as a cursor about a display, and *Dragging* (State 2) for moving an icon on the desktop or for grouping a set of objects or a range of text. The model seems simple and obvious, and we might question its ability to add significant insight to the existing body of pointing-device research. Yet, the model can be extended to capture additional aspects of pointing-device interaction such as multibutton interaction, stylus input, and direct versus indirect input. See Buxton (1990) for further details.

As further evidence of the utility of Buxton's work, we recently used the 3-state model to characterize an interaction technique that didn't even exist at the time the model was introduced. The insight gained led to a redesign of an interaction technique. This work is briefly recounted here. Never shy of innovation, Apple Computer, Inc. (Cupertino, CA) took a bold step in 1994 by commercializing a new pointing device in its PowerBook 500 notebook computer: the TrackPoint touchpad (MacNeill & Blickenstorfer, 1996). And the rest is history. Today, touchpads are the dominant pointing device for notebook computers. (IBM and Toshiba are notable holdouts, preferring isometric joysticks instead.) One of the interaction techniques supported by touchpads is *lift-and-tap*, wherein primitives like clicking, double-clicking, or dragging are implemented without a button. These new interaction primitives are easily represented by Buxton's 3-state model. Figure 3.2 provides a simple comparison of the state transitions for dragging tasks using a mouse and using lift-and-tap on a touchpad.

Diagrams like these are evidence of the descriptive power of models such as Buxton's. Two observations follow: (1) Lift-and-tap necessitates extra state transitions in comparison to a mouse, and (2) the use of state 1–0–1 transitions for lift-

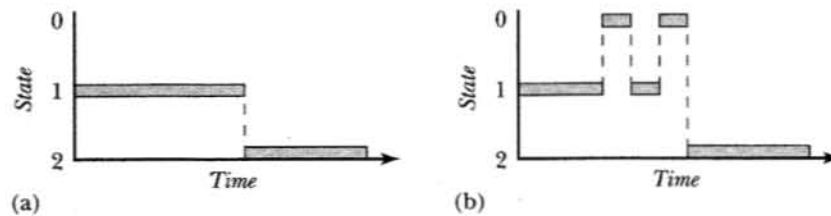


FIGURE
3.2

Dragging tasks: (a) mouse; (b) lift-and-tap touchpad. Dragging begins upon entering State 2 (after MacKenzie & Oniszczak, 1997).

and-tap is confounded with clutching (not shown), which uses the same state transitions.

Among users' frustrations in using touchpads is that these primitives are difficult, awkward, or error prone. For example, if a finger touch following a lift is spatially displaced from the point of lifting, the system often incorrectly enters the Tracking state (State 1) instead of the Dragging state (State 2).

Armed with a deeper understanding of touchpad interaction, we undertook a redesign of the buttonless implementation of state transitions on touchpads. The additional pressure-sensing capability of touchpads was used to implement State 1–2 transitions by "pressing harder." A built-in relay provided both tactile and aural feedback to inform the user of state transitions, much like the feedback in pressing a mouse button. Thus, clicking, double clicking, and dragging were implemented on the touchpad, without a button, yet using the same state transitions as on a mouse. The complete details are presented in the 1997 and 1998 proceedings of the Association for Computing Machinery's (ACM) Special Group for Computer-Human Interaction (SIGCHI) (MacKenzie & Oniszczak, 1997; MacKenzie & Oniszczak, 1998).

Model for Mapping Degrees of Freedom to Dimensions

One final example of a descriptive model will be given. A topic of great interest in HCI is interactive graphics. The vocabulary of such systems includes terms like *dimensions* and *degrees of freedom*. These terms are often used interchangeably; however, the terms are different, and the distinction between them is often poorly articulated. Before proceeding, Figure 3.3 is necessary to establish some terminology.

The figure shows a mousepad and identifies the most common labels for the axes of motion. Left-to-right motion is *translational*, along the X axis; to-and-fro

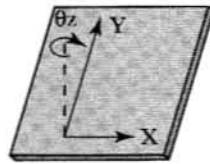


FIGURE
3.3

A mousepad is a two-dimensional surface with three degrees of freedom: X, Y, and θ_z .

Dimensions	Degree of freedom	Mouse	Two-ball mouse	Other			
3D 2D {	X	●	●	●	●	●	●
	Y	●	●	●	●	●	●
	θ_z	○	●	○	●	○	●
	Z	○	○	●	●	○	○
	θ_x	○	○	○	●	●	●
	θ_y	○	○	○	●	●	●

FIGURE
3.4

Mapping degrees of freedom to dimensions (see text for discussion).

motion is translational along the Y axis. If the mouse is rotated on the pad, motion is *rotational* about the Z axis, labeled θ_z in the figure. Although most users are quick to declare the mouse a two-dimensional (2D) device (e.g., Zhai & MacKenzie, 1998), this is not quite true. Here's where we see a slight point of confusion between dimensions (D) and degrees of freedom (DOF). A true 2D device has three degrees of freedom (3-DOF), as illustrated in Figure 3.3. If a mouse were a true 2D device, then translational motion along the X and Y axes would be sensed, *as well as rotational motion about the Z axis*. Of course, rotational motion of the mouse is not sensed. This deficiency is illustrated in Figure 3.4. The left two columns show our descriptive model for mapping degrees of freedom to dimensions, using the terminology in Figure 3.3. The third column shows the mappings implemented for a conventional mouse.

The model in Figure 3.4 is a simple tool for illustrating the mappings of degrees of freedom to dimensions for real or hypothetical input devices. The

apparent shortcoming of the standard mouse as a 2D input device motivated us to design a new mouse to implement the missing DOF. As shown in the fourth column in Figure 3.4, our two-ball mouse also senses rotational motion about the Z axis. An object acquired in a graphics application with this mouse can be positioned in a single gesture both translationally along the X and Y axis and rotationally about the Z axis, thus negating the need for a "rotate tool." See MacKenzie, Soukoreff, and Pal (1997) for details.

Figure 3.4 also shows the mappings of degrees of freedom to dimensions for other pointing devices. Column A identifies devices that sense only translation along the three axes. An example is the Owl by Pegasus Technologies, Ltd. (Tel Aviv, Israel). Column B identifies true 3D input devices, such as trackers used in virtual reality systems. Such devices sense X-, Y-, and Z-axis translation and rotation. An example is the Isotrak II by Polhemus, Inc. (Colchester, VT). Column C identifies an interesting research prototype known as the Rockin'Mouse. It looks and feels like a mouse, but it has a curved bottom and senses both translation and rotation about the X and Y axes (see Balakrishnan, Baudel, Kurtenbach, & Fitzmaurice, 1997, for details). Column D identifies a hypothetical 5-DOF input device. Because it does not sense Z-axis translation, the device is operated on the mousepad and therefore has that special mouselike appeal.

3.3 SCIENTIFIC FOUNDATIONS AND MODEL DESCRIPTIONS

The two models chosen for detailed study in this chapter are Fitts' law and Guiard's model of bimanual skill. As we shall see, Fitts' law falls within the scope of a predictive model, whereas Guiard's model is an example of a descriptive model.

3.3.1 Fitts' Law

Fitts' Law is one of the most robust and highly adopted models of human movement. The model is, arguably, the most successful of many efforts to model human behavior as an information-processing activity. (For detailed reviews, see MacKenzie, 1991; MacKenzie, 1992; Meyer, Smith, Kornblum, Abrams, & Wright, 1990; Welford, 1968). Fitts was interested in applying information theory to measure the difficulty of movement tasks and the human rate of information processing as tasks are realized. He argued that the amplitude of an aimed movement was analogous to an electronic signal and that the spatial accuracy of the

move was analogous to electronic noise. Furthermore, he proposed that the human motor system is like a communications channel, wherein movements are viewed as the transmission of signals. His analogy is based on Shannon's Theorem 17, expressing the effective information capacity C (in bits/s) of a communications channel of bandwidth B (in s^{-1} or Hz) as

$$C = B \log_2(S / N + 1) \quad (3.3)$$

where S is the signal power and N is the noise power (Shannon & Weaver, 1949, pp. 100–103).

Fitts presented his analogy—now his “law”—in two highly celebrated papers, one in 1954 (Fitts, 1954), the second in 1964 (Fitts & Peterson, 1964). The 1954 paper described a serial, or reciprocal, target-acquisition task, wherein subjects alternately tapped on targets of width W separated by amplitude A ; see Figure 3.5(a). The 1964 paper described a similar experiment using a discrete task, wherein subjects selected one of two targets in response to a stimulus light; see Figure 3.5(b). It is easy to imagine how the mechanical apparatus Fitts used can be replaced by computer input devices and targets rendered on a cathode-ray tube (CRT) display.

Fitts proposed to quantify a movement task's difficulty— ID , the *index of difficulty*—using information theory by the metric “bits.” Specifically,

$$ID = \log_2(2A / W) \quad (3.4)$$

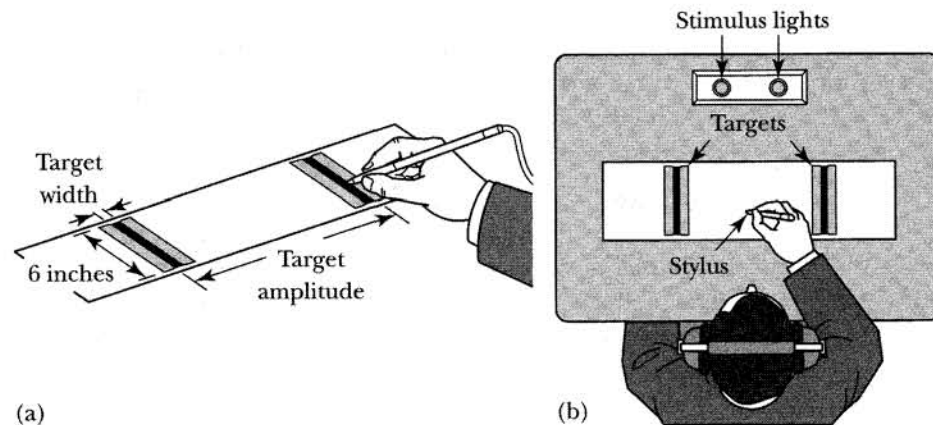


FIGURE
3.5

Experimental paradigm for Fitts' Law: (a) serial task (after Fitts, 1954); (b) discrete task (after Fitts & Peterson, 1964).

The amplitude (A) and width (W) in Equation 3.4 are analogous to Shannon's signal (S) and noise (N) in Equation 3.3. The following expression for ID was introduced to improve the information-theoretic analogy (MacKenzie, 1989):

$$ID = \log_2(A / W + 1) \quad (3.5)$$

Because A and W are both measures of distance, the term within the parentheses in Equation 3.5 is without units. The unit *bits* emerges from the somewhat arbitrary choice of base 2 for the logarithm. From Equation 3.5, the movement time (MT) to complete a task is predicted using a simple linear equation:

$$MT = a + b \times ID \quad (3.6)$$

In building a Fitts' Law model, the slope and intercept coefficients in the prediction equation are determined through empirical tests, typically using linear regression. The tests are undertaken in a controlled experiment using a group of subjects and one or more input devices and task conditions.

The design of experiments for Fitts' Law studies is straightforward. Tasks are devised to cover a range of difficulties by varying A and W . For each task condition, multiple trials are conducted, and the time to execute each is recorded and stored electronically for statistical analysis. Accuracy is also recorded, either through the x-y coordinates of selection or through the error rate—the percentage of trials selected with the cursor outside the target. Example data sets for two devices tested in a serial task are given in Figure 3.6. For the purpose of this discussion, the devices are simply referred to as “A” and “B.”¹ The figure shows the controlled variables A and W , along with the computed index of difficulty (ID , see Equation 3.5) and, for each condition, the observed error rate (ER) and movement time (MT). Each dependent measure is based on 240 observations (12 participants $\times$ 20 trials per condition).

Figure 3.7 shows a common reporting technique for Fitts' Law experiments. The nine $MT-ID$ points for each device in Figure 3.6 are plotted along with the regression lines. The prediction equations for MT as a function of ID are also shown, as are the R^2 statistics for the percentage of variance in the observations explained by the model. It is noteworthy of Fitts' Law in general that R^2 is

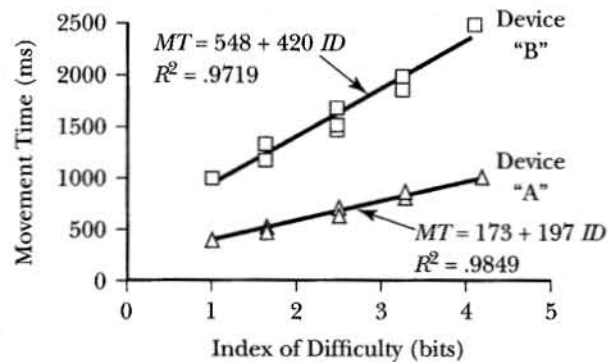
¹ The data sets in Figure 3.6 are from the fourth block of trials in a Fitts' Law experiment comparing four pointing devices, including a Microsoft Mouse 2.0 (Device “A”) and an Interlink RemotePoint (Device “B”). See MacKenzie and Jusoh (2001) for complete details on the devices and the experimental methodology.

A (pixels)	W (pixels)	ID (bits)	Device "A"		Device "B"	
			ER (%)	MT (ms)	ER (%)	MT (ms)
40	10	2.32	2.08	665	1.25	1587
40	20	1.58	3.33	501	2.08	1293
40	40	1.00	1.25	361	0.42	1001
80	10	3.17	2.92	762	2.08	1874
80	20	2.32	1.67	604	2.08	1442
80	40	1.58	1.67	481	0.83	1175
160	10	4.09	3.75	979	2.08	2353
160	20	3.17	5.42	823	1.67	1788
160	40	2.32	4.17	615	0.83	1480
Mean:		2.40	2.92	644	1.48	1555

FIGURE

Example data sets for two devices from a Fitts' Law experiment.

3.6



FIGURE

Scatter plots and regression lines for data sets in Figure 3.6.

3.7

usually very high, attesting to the strength of the model as a predictor for rapid aimed movements. For Device "A," for example, the regression equation explains about 98% of the variance in the observed movement times.

Because task difficulty is analogous to information, the rate of task execution is interpreted as the human rate of information processing. For example, if a

task rated at, say, $ID = 4.09$ bits is executed in $MT = 979$ ms, then the human rate of information processing for that task is $4.09/0.979 = 4.18$ bits/s. In fact, this very example is seen in Figure 3.6 for Device "A" for the condition $A = 160$ and $W = 10$. Fitts called this measure the *index of performance (IP)*, although the term *throughput (TP)* is more common today (Douglas, Kirkpatrick, & MacKenzie, 1999; ISO, 1999).

More likely, the mean throughput is of interest. Reading from the bottom row in Figure 3.6, the mean throughput across all the A - W conditions for Device "A" is $TP = 2.40/0.644 = 3.73$ bits/s. For Device "B," $TP = 2.40/1.555 = 1.57$ bits/s. Using throughput as a criterion, therefore, we conclude that users' performance with Device "A" was about $3.73/1.57 = 2.4$ times better than performance with Device "B."

Unfortunately, the computation of throughput is not as straightforward as just described. In many studies it is reported as the reciprocal of the slope of the regression line (e.g., Card, English, & Burr, 1978; MacKenzie, Sellen, & Buxton, 1991). By this reasoning and from Figure 3.7, we see that throughput for Device "A" is $TP = 1/0.197 = 5.08$ bits/s. This figure is a nontrivial 36% higher than $TP = 3.73$ bits/s, computed from the same data except dividing the mean ID by the mean MT . The computation of TP is further complicated by the need to include spatial variability (namely accuracy) to further improve the analogy to Shannon's theorem (Fitts & Peterson, 1964; Welford, 1968). This is accomplished by replacing the target width (W) in Equation 3.5 with the "effective target width" (W_e), computed as follows:

$$W_e = 4.133 \times SD_x \quad (3.7)$$

where SD_x is the standard deviation in the selection coordinates computed over a block of trials.² If only discrete error rates are recorded, as shown in Figure 3.6, then an alternate technique is used to compute W_e (see MacKenzie, 1992, for details). Throughput so computed is extremely useful because in a single measure it captures both speed and accuracy in performance, thus increasing the ability of the measure to reveal overall performance differences between devices or interaction techniques.

If the primary goal in a Fitts' Law experiment is to establish performance differences between devices and/or interaction techniques, then throughput is

² The coefficient 4.133 emerges from the term $(2 \times \pi \times e)^{1/2}$ in Shannon's original theorem. See MacKenzie (1992) for details.

the best choice as the criterion measure. The recommended calculation is the division of means with the aforementioned adjustment for accuracy:

$$TP = ID_e / MT \quad (3.8)$$

where

$$ID_e = \log_2(A / W_e + 1) \quad (3.9)$$

This is the formula specified in the ISO standard for evaluating computer pointing devices (ISO, 1999). If the primary goal in a Fitts' Law experiment is to generate a movement time-prediction equation, then Equation 3.6 should be used. We will put this equation to work later in a case study.

The first use of Fitts' Law in HCI was the study by Card et al. (1978), in which they empirically established the superiority of the mouse over an isometric joystick. Many other Fitts' Law studies have appeared since (e.g., Accot & Zhai, 1999; Arsenault & Ware, 2000; Douglas et al., 1999; Epps, 1986; MacKenzie, Kauppinen, & Silfverberg, 2001; MacKenzie & Oniszczak, 1998).

This concludes our detailed analysis of Fitts' Law. Later, in Section 3.4, we present a case study showing an application of Fitts' Law in predicting text-entry rates on mobile phones.

3.3.2 Guiard's Model of Bimanual Skill

Humans are not only two handed, they also use their hands differently. This fact has been the subject of considerable study in a specialized area of motor behavior known as *bimanual control* or *laterality* (Kelso, Southard, & Goodman, 1979; Peters, 1985; Porac & Coren, 1981; Wing, 1982). Studying the between-hand division of labor in everyday tasks reveals that most tasks are asymmetric: Our hands have different roles and perform distinctly different tasks. Given this, and the knowledge that people are either right handed or left handed, it follows that an examination of the assignment of tasks to hands is a useful exercise. This is the essence of Guiard's work, as presented in a seminal paper in 1987 (Guiard, 1987). The result of this work is a descriptive model of bimanual skill. Although the model lacks the mathematical rigor of Fitts' Law, it is by no means less valuable as a tool to aid and inform designers of interactive systems.

An important goal for descriptive models is to delineate the critical characteristics in a problem space, and this is precisely what Guiard's model does.

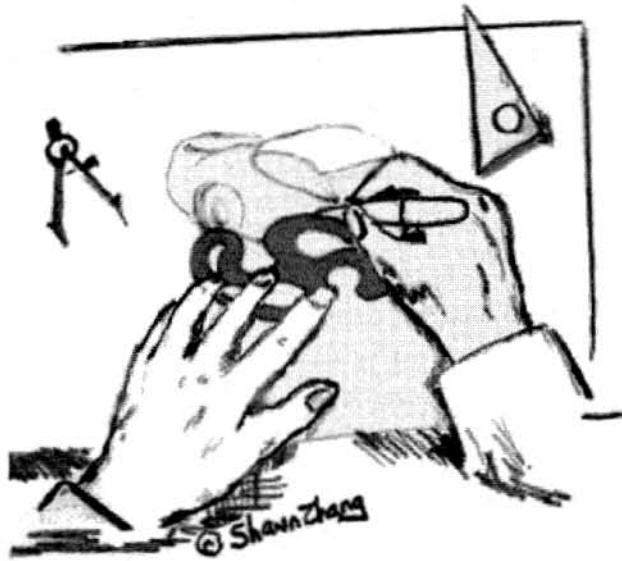
Hand	Role and Action
Nonpreferred	♦ leads the preferred hand ♦ sets the spatial frame of reference for the preferred hand ♦ performs coarse movements
Preferred	♦ follows the nonpreferred hand ♦ works within the established frame of reference set by the nonpreferred hand ♦ performs fine movements

FIGURE 3.8 Guiard's model of bimanual control.

Figure 3.8 identifies the roles and actions of the nonpreferred and preferred hands, as given by Guiard.

The best way to explain the points in Figure 3.8 is through an exemplary illustration and narrative. In Figure 3.9, a right-handed graphic artist is shown sketching the design of a new car. The artist acquires the template with her left hand (*nonpreferred hand leads*). The template is manipulated over the workspace (*coarse movement, sets the frame of reference*). The stylus is acquired in the right hand (*preferred hand follows*) and brought into the vicinity of the template (*works within frame of reference set by the nonpreferred hand*). Sketching takes place (*preferred hand makes precise movements*).

The roles and actions just described provide a provocative and fresh way of describing how humans approach common tasks. This is true both for common tasks and in the narrower context of human-computer interaction. About the same time that Guiard's model appeared, Buxton and Myers (1986) conducted an experiment to investigate two-handed input in computer-aided design (CAD) systems. Their work was undertaken independently of Guiard's work or of other work in experimental psychology on bimanual control or laterality. One of their studies describes a positioning and zooming task where the subtasks were performed by separate hands. The positioning task was performed using a graphics tablet "puck" in the preferred hand, while the zooming task was performed using a slider to control the size of an object. The goal was to manipulate an object so that it coincided in position and size with a similar object. Their second study describes a scrolling-and-selection task. The goal was to select words in a document. The document was sufficiently large that scrolling was necessary to locate the desired words. Their key findings were that subjects—without being instructed—gravitated to a two-handed interaction style, and that task-completion times were smaller when a two-handed strategy was employed.



FIGURE

Two-handed interaction paradigm. (Sketch courtesy of Shawn Zhang.)

3.9

Unfortunately, Buxton and Myers (1986) provided very little insight into the differential roles of the hands in bimanual tasks (cf. Figure 3.8). They concluded only that the subjects' natural tendency toward a two-handed strategy was due to the "efficiency of hand motion" (p. 321). Watershed moments in multidisciplinary fields like HCI often occur when researchers through due diligence locate and adopt relevant research in other fields—research that can inform and guide their own discipline.³ The union of HCI research in two-handed input and basic research in bimanual control occurred several years later, through the efforts of Paul Kabbash (Kabbash, Buxton, & Sellen, 1994; Kabbash, MacKenzie, & Buxton, 1993). The paper by Kabbash, Buxton, and Sellen (1994) was the first in HCI to cite Guiard's 1987 paper and to be guided by this important model. Subsequent HCI research in two-handed interaction has consistently used Guiard's descriptive model (see Figure 3.8) as a guiding principle in designing new interactive techniques that appropriately engage the hands of users.

³ There are many examples in HCI, for example, Card, English, and Burr's (1978) first use of Fitts' Law (Fitts, 1954), or Norman's (1988) introduction of Gibson's affordances (Gibson, 1979).

3.4 CASE STUDIES

In this section, two case studies are presented. The first is an application of Fitts' Law in predicting text-entry rates on mobile, or cell, phones. The second is an examination of the affordances of the desktop computer interfaces in the context of two-handed input.

3.4.1 Case Study #1: Fitts' Law Predictions of Text-Entry Rates on Mobile Phones

One recent application of Fitts' Law is in predicting text-entry rates on mobile phones. This is particularly relevant today because of the tremendous demand for text messaging on mobile phones. Current volumes are on the order of billions of SMS (short message service) messages per month (www.gsmworld.com).

Text Entry on Mobile Phones

Before presenting our model, the main techniques for entering text on a mobile phone are described. Most mobile phones include a 12-key keypad similar to that in Figure 3.10. The keypad consists of number keys 0–9 and two additional keys (# and *). Characters *a–z* are spread over keys 2–9 in alphabetic order, with the SPACE character assigned to the 0 key. Because there are fewer keys than the 26 needed for the characters *a–z*, three or four characters are grouped on each key, and, so, ambiguity arises. There are two main approaches to text entry on a phone keypad: the *multitap* method and the *one-key with disambiguation* method.

Multitap. The multitap method is currently the most common text input method for mobile phones. With this approach, the user presses each key one or more times to specify the input character. For example, the 2 key is pressed once for the character *a*, twice for *b*, and three times for *c* (see Figure 3.10). As an example, *quick brown fox* is entered as follows:

```
77 88 444 222 55 0 22 777 666 9 66 0 333 666 99
q u i c k _ b r o w n _ f o x
```

Clearly, the multitap method bears substantial overhead. In this example, 33 key presses generated 15 characters of text. Calculations using a language corpus



FIGURE 3.10 Standard 12-key mobile phone keypad.

indicate that “on average” the multitap method requires 2.034 keystrokes per character when entering English text (MacKenzie, 2002).⁴

The multitap approach suffers from the additional problem of segmentation. When a character is on the same key as the previous character, further ambiguity arises. For example, both letters in the word *on* are on the 6 key (see Figure 3.10). Successive key presses are segmented (viz., assigned to different characters) using either a timeout or a dedicated timeout kill key. We will say more about these techniques shortly.

One-key with disambiguation. An alternative way to overcome the ambiguity problem is to add linguistic knowledge to the system. We call this technique *one-key with disambiguation*. An example is *T9* by Tegic Communications, Inc. (Seattle, WA; www.tegic.com/). When using *T9*, each key is pressed only once. For example,

```
7 8 4 2 5 0 2 7 6 9 6 0 3 6 9
q u i c k _ b r o w n _ f o x
```

Although each key maps to more than one letter, the system performs beyond-the-scenes disambiguation in determining the intended word. Naturally, linguistic disambiguation is not perfect, since multiple words may have the same key

⁴ This figure is computed based on entering text consisting of the 26 letters plus the SPACE character. If only letters are considered, the figure is about 2.26 keystrokes per character.

sequence. In these cases, the most common word is the default. A demonstration follows (presses of the 0 key are omitted for clarity):

```
843 78425 27696 369 58677 6837 843 5299 364
the quick brown fox jumps over the jazz dog
tie stick crown lumps muds tie lazy fog
vie
```

Candidate words are shown from top to bottom, with the most probable at the top. Of the nine words in the phrase, eight are ambiguous, given the required key sequence. For seven of the eight, however, the most probable word is the intended word. The most probable word is *not* the intended word in just one instance, with *jazz* being more probable in English than *lazy*. In this case, the user must press additional keys to obtain the desired word.

Given the two input methods just described, it is worth speculating on the relative text-entry rates attainable with each. For this, we can develop a prediction model using Fitts' Law in combination with the probabilities of letter pairs, or *digrams*, in the desired language. This is described next.

Predicting Text-Entry Rates Using Fitts' Law

Fitts' Law is not an appropriate model for predicting typing speeds on a Qwerty keyboard, because the task is complex and involves parallel and/or overlapping movements by two hands and ten fingers. If the task is reduced to input via a single finger or stylus, however, then the situation is completely different. With a single stream of motor actions, the task is a standard target-acquisition task conforming to the Fitts paradigm.

Text entry on a mobile phone, for example, consists of aiming for and acquiring (viz., pressing) a series of keys "as quickly and accurately as possible." Users typically perform this task either using their index finger or thumb. The time to press any key, given any previous key, can be readily predicted using Fitts' Law. Models for pressing keys on a mobile phone are reported by Silfverberg, MacKenzie, and Korhonen (2000). For index-finger input,

$$MT = 165 + 52 ID \quad (3.10)$$

and for thumb input,

$$MT = 176 + 64 ID \quad (3.11)$$

The only ingredients required to build a text-entry prediction model are information on the position and size of keys, the letter assignment to keys, and the relative probabilities of digrams in the target language. These are close at hand. The position and size of keys are easily quantized for any mobile-phone keypad using a ruler. The standard assignment of letters to keys is shown in Figure 3.10. Tables of digram probabilities are available from a variety of sources (e.g., Mayzner & Tresselt, 1965; Soukoreff & MacKenzie, 1995; Underwood & Schulz, 1960). A final task—laborious but simple—is to put this information together along with the Fitts' Law equations in an electronic form suitable for generating the predictions. A spreadsheet application suffices.

If we limit the model to text entry using 26 letters plus the SPACE character, then there are $27^2 = 727$ target acquisition tasks, one for each letter j given a preceding letter i . The time to enter each i - j sequence is predicted using Fitts' Law, yielding MT_{ij} . The time is weighted by the probability of the digram in the target language, P_{ij} . For English, sequences like *t-h* or *e-SPACE* have high probabilities, whereas sequences like *g-k* or *f-v* have low probabilities. The sum of the 727 weighted-movement times is the mean movement time per character for the target language for the given interaction technique:

$$MT_L = \sum \sum (P_{ij} \times MT_{ij}) \quad (3.12)$$

This is converted to a text-entry rate in words per minute (wpm) by multiplying by 60 seconds per minute and dividing by 5 characters per word⁵:

$$WPM = MT_L \times (60 / 5) \quad (3.13)$$

Predicted Text-Entry Rates on Mobile Phones

There are a few additional details to accommodate before predictions are available. For the multitap method, keys are pressed one to four times depending on each letter's position on a key. To account for this behavior, we introduce the key-repeat time, MT_{REPEAT} , the time to press a key having just pressed the same key. This is a Fitts' Law task with "zero movement amplitude" (viz., $A = 0$). From Equation 3.5, the index of difficulty is $ID = \log_2(0/W + 1) = 0$ bits. Hence, MT_{REPEAT} is the intercept in the Fitts' Law (see Equations 3.6, 3.10, and 3.11). Thus, to MT_{ij} we add $N \times MT_{REPEAT}$, where N varies from 0 to 3.

⁵ The average size of a word in English is about 4.5 characters. However, in computing text-entry throughput in "words per minute," it is customary to consider a "word" any sequence of 5 characters, including letters, spaces, punctuation, and so forth.

Method	Predicted Expert Entry Rate (wpm)	
	Index Finger	Thumb
Multitap		
wait for timeout	22.5	20.8
timeout kill	27.2	24.5
One-key with disambiguation	45.7	40.6

FIGURE 3.11 Expert text-entry rate predictions (wpm) for mobile phones (from Silfverberg et al., 2000).

Additionally, the multitap method requires a special technique to segment consecutive characters on the same key, as noted earlier. One technique—*wait-for-timeout*—is to hesitate for a timeout interval (T_{TIMEOUT}) between keypresses for each character. Although durations vary among manufacturers, a reasonable value is $T_{\text{TIMEOUT}} = 1500$ ms. This is the setting on Nokia phones. The other technique—*timeout-kill*—is to press an explicit timeout kill key, thus allowing direct entry of the second character. On Nokia phones, this is implemented using the DOWN-ARROW key. For the timeout-kill strategy, we use $MT_{\text{FKILL}} + MT_{\text{KILL},j}$ instead of MT_{ij} , where i and j are on the same key. The user may choose whether to use the wait-for-timeout or timeout-kill segmentation strategy; therefore, a prediction model is built for each strategy. Without further ado, Figure 3.11 gives the predicted expert text-entry rates for mobile phones for the input methods and interaction techniques just described.

Multitap entry rates vary from 20.8 wpm to 27.2 wpm depending on interaction technique (index finger vs. thumb) and the timeout strategy employed. In general, index-finger entry is faster than thumb entry.

A few follow-up comments are warranted. Note the prefix “expert” in Figure 3.11. Indeed, the predictions represent an upper bound. In particular, they do not account for the time to scan the keypad visually to find the next key to press. Zero visual-scan time is reasonable for expert users, but novices must visually attend to the keypad during entry. This will push entry rates down. The predictions also assume error-free input. This simplifies the prediction model, but, in practice, users are fallible and overhead is incurred in correcting mistakes. So, all rates in Figure 3.11 must be viewed in the proper context: They represent only the motor component of error-free expert input.

With predictions above 40 wpm, clearly the one-key-with-disambiguation method is far superior to the multitap method. However, these rates are coincident with rather broad assumptions, and they also must be viewed in the

appropriate context. As noted, the rates are for error-free expert input. However, at least two additional assumptions are present for the one-key-with-disambiguation method: (1) all words are in the system's dictionary, and (2) when ambiguity arises, the intended word is the most probable in the list of candidates. Clearly, violations in these assumptions will push rates down, but a detailed discussion of the mechanisms at work is beyond the scope of this chapter. See Silfverberg et al. (2000) for further discussion.

For our next case study, we return to Guiard's model of bimanual control.

3.4.2 Case Study #2: Bimanual Control and Desktop Computer Affordances

Affordances are the action possibilities an artifact offers to a user (Gibson, 1979). A keyboard's keys can be pressed, a mouse can be acquired and moved, mouse buttons can be pressed, and a mouse wheel can be rotated. These are the *what* of desktop computer affordances. Let's consider the *where*.

On a typical desktop keyboard, keys are spread across the device, there is a wide space bar along the bottom, and nonalpha keys are in various locations. Although slight variations exist, the 101-style keyboard is the most common today. An example is given in Figure 3.12, which also identifies the categories of keys as per the key-action model presented earlier.

Of concern here are the "power keys," consisting of *executive keys* (e.g., ENTER) and *modifier keys* (e.g., SHIFT). On the keyboard's left, we find SHIFT, CTRL, ALT, TAB, CAPS LOCK, and ESC, while on the right we find no less than 18 power keys: SHIFT, CTRL, ALT, ENTER, BACKSPACE, INSERT, DELETE, HOME, END, PAGE UP, PAGE DOWN, ←, ↑, →, ↓, PRNT SCR, SCROLL LOCK, and PAUSE. Because SHIFT, CTRL, and ALT are mirrored, they do not pose a left- or right-hand bias and are eliminated from further discussion. Only three keys on the left (ESC, TAB, CAPS LOCK) are without a right-side replica; thus, the number of keys are 3 on the left and 15 on the right. These are identified by dots in Figure 3.12. The ratio is 1:5.

Clearly, the keyboard is entrenched with a right-side bias. This was fine in the 1970s; however, the emergence of the graphical user interface (GUI) and point-and-click interfaces in the 1980s changed everything (but not the keyboard, apparently). Most users grasp the mouse in their right hand. A simple observation is that the right hand is busy and, arguably, overloaded. Interactions that involve both power-key activation and pointing are exacerbated for right-handed users: Their options are to "reach over" with the left hand, or to release the mouse and acquire and activate a power key with their right hand.

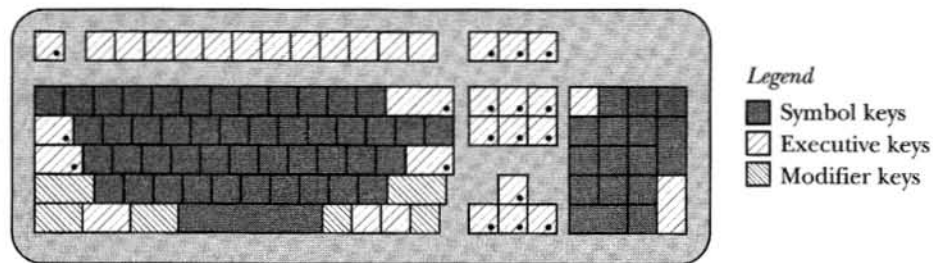


FIGURE 3.12 101-style keyboard and key categories as per the key-action model (see text for a discussion).

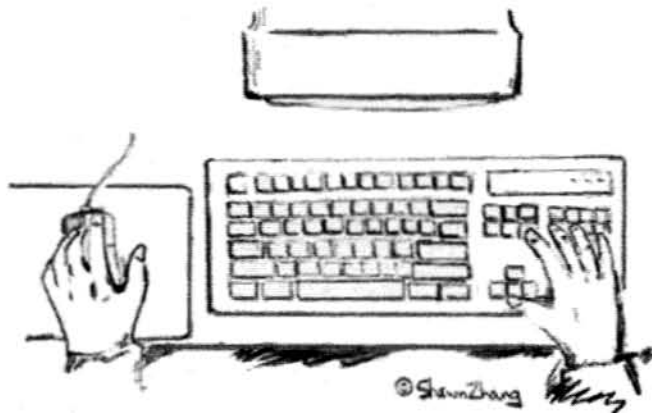


FIGURE 3.13 A left-handed user interacting with a desktop system. (Sketch courtesy of Shawn Zhang.)

The scenario differs for left-handed users, however. When they position the mouse on the left side of the keyboard and manipulate it with their left hand, the action possibilities of the desktop interface are dramatically changed (see Figure 3.13). The confounding of power-key activation with pointing operations is greatly reduced.

This brief analysis clearly shows that the affordances of a typical desktop system are substantially different for left-handed users who manipulate the mouse in their left hand. An important question is this: Are left-handed users well-served by the configuration in Figure 3.13? This question is by no means simple to answer. A detailed response would require a checklist-style examination of the

myriad tasks typical of desktop use. However, a cursory answer to this question is “yes.” Let’s examine why.

If we combine the earlier points on affordances and human skill in analyzing common GUI tasks, we see that the bimanual skill model is often violated (viz., preferred hand leads). Of course, there is little gained in suggesting a ground-up redesign of computer keyboards—it’s too late! Faced with the unyielding affordances of current keyboards, users (both left handed and right handed) naturally discover and adopt strategies of optimization. In short, users find ways to optimize, often in spite of the interface. The path of discovery seems to favor left-handed users, however. Four examples are cited in Figure 3.14, as performed by left-handed users.

Although multiple techniques are available for the tasks described in Figure 3.14, the example tasks are not only fast for left-handed users, they are *faster* for left-handed users than for right-handed users. (Right-handed readers may wish to reconsider the examples.) The unifying observation is this: *When pointing is juxtaposed with power-key activation (excluding SHIFT, ALT, and CONTROL), the desktop interface presents a left-hand bias.* This is evident in the Figure 3.14. All the tasks involve a tight coupling of pointing operations with power-key activation. If right-handed users attempt the same tasks, their options are to release the mouse and activate the power key with their right hand, or to reach over the keyboard with their left hand—awkward in either case.

Importantly, the tasks are what we do *all the time*. Note in the third column in Figure 3.14 that trailing movements are also identified as “overlapping.” Because the subtasks in the second and third columns are performed by separate hands, there is preparatory trailing movement that overlaps the leading movement. The action point in the trailing movement *immediately* follows the leading movement. And so, left-handed users persistently “cash in” on small-time savings. This leads to a simple conclusion that the desktop interface presents a left-hand bias.

Scrolling

Although Guiard’s model of bimanual control alerts us to the differential roles of our hands, the analysis just presented is after the fact. It is unlikely that keyboard manufacturers will launch a redesign of the keyboard—positioning power keys on the left to facilitate right-handed interaction—based on the examples in Figure 3.14. However, Guiard’s model does lend credence to one design deficiency worthy of correcting—scrolling. Although scrolling is an extremely common GUI task, affordances are poor for left-handed users *and* right-handed users.

Scrolling is traditionally accomplished by dragging the “elevator” of the scrollbar positioned along the right-hand side of an application’s window.

Task	Leading Movement	Trailing/Overlapping Movement
Delete	<i>Left hand</i> —manipulate pointer with mouse and select text/object by double clicking or dragging	<i>Right hand</i> —press DELETE (probably with little finger)
Select an option in a window (see Figure 3.15)	<i>Left hand</i> —manipulate pointer with mouse and click on an option	<i>Right hand</i> —press ENTER (Note: OK button is the default; see Figure 3.15)
Click on a link in a browser	<i>Right hand</i> —navigate to link via PAGE UP and/or PAGE DOWN keys	<i>Left hand</i> —manipulate pointer with mouse and select link by clicking on it
Open a file, open a folder, or launch a program	<i>Left hand</i> —manipulate pointer with mouse and single click on an icon	<i>Right hand</i> —press ENTER (Note: avoids error-prone double-click operation)

FIGURE 3.14 Examples of common tasks, as performed by a left-handed user manipulating the mouse in the left hand.

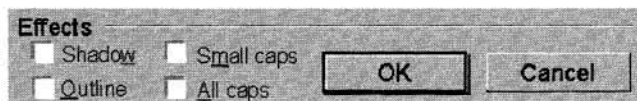


FIGURE 3.15 Selecting an option in a window (see Figure 3.14).

Acquiring the elevator is a target-acquisition task taking up to two seconds per trial. More important, this action is in conflict with a basic goal of good user interfaces: unobtrusiveness and transparency. That is, users should not be required to divert their attention from the primary task (reading, editing, etc.) to explicitly manipulate user-interface widgets (Zhai & MacKenzie, 1998).

This we see as an opportunity for design. Scrolling is well suited to delegation to the nonpreferred hand, since its relationship with other tasks conforms to Guiard's model of bimanual control. Evidence of this is presented in Figure 3.16, which contrasts the properties of scrolling with tasks typically performed in concert with scrolling. (The reader is invited to compare the first three bullets in Figure 3.16 with the first three bullets in Figure 3.8, which presents the guiding principles in Guiard's model of bimanual control.)

Desktop affordances for scrolling changed dramatically in 1996 with the introduction of Microsoft's IntelliMouse, which included a scrolling wheel between the mouse buttons. Numerous copy-cat variations have since appeared from other manufacturers. The so-called wheel mouse puts scrolling in the

Task	Characteristics
Scrolling	† precedes/overlaps other tasks † sets the frame of reference † minimal precision needed (coarse)
Selecting, editing, reading, drawing, and so on.	† follows/overlaps scrolling † works within frame of reference set by scrolling † demands precision (fine)

FIGURE 3.16 Relationship between scrolling and common GUI tasks.



FIGURE 3.17 Scrolling interface example. (Sketch courtesy of Shawn Zhang.)

preferred hand. This is bad news for left-handed users, but it is even worse for right-handed users because of the higher right-side demands presented by the power-key bias noted earlier.

Figure 3.17 presents a scrolling concept for a right-handed user. A touch strip is shown, but a wheel could just as easily be used. There are many implementation issues, such as scrolling sensitivity and support for up/down paging, but space precludes elaboration. Our point is simply that scrolling should be delegated to the nondominant hand. See also Buxton and Myers (1986), Kabbash et al. (1994), and MacKenzie and Guiard (2001).

In this section, we examined common GUI tasks and patterns of bimanual skill. Two important observations emerged: first, current desktop systems are

biased for left-handed users; and second, scrolling should be delegated to the nondominant hand. In the absence of a detailed functional taxonomy unifying the actions of the preferred hand and nonpreferred hand with system affordances, however, further design recommendations are premature. Broadly stated, such efforts should seek to tightly couple the hands—supporting sequential, cooperative action—with tasks such as power-key activation, pointing, selecting, and scrolling.

3.5 CURRENT STATUS AND FURTHER READING

Both Fitts' Law and Guiard's model of bimanual skill continue as popular research tools in human-computer interaction. Fitts' Law remains the preeminent model for pointing-device research. Evidence of this is the recent appearance of Fitts' Law in an ISO standard. The full standard, ISO 9241, is titled *Ergonomic requirements for office work with visual display terminals (VDTs)*. Part 9 is titled *Requirements for non-keyboard input devices* (ISO, 1999). ISO 9241-9 is a comprehensive document that defines evaluation procedures for measuring user performance, comfort, and effort with pointing devices. An experimental protocol defines subject samples, stimuli, experimental design, environmental conditions, furniture adjustments, data collection procedures, and data-analysis recommendations.

Performance is measured on any of six exemplary tasks, three of which are standard point-select tasks conforming to the Fitts' paradigm. The dependent measure for the evaluations is throughput, computed as noted earlier (see Equation 3.8). The use of ISO 9241-9 should go a long way in bringing consistency to empirical evaluations of computer pointing devices. In particular, across-study comparisons are strengthened if studies adopt a consistent methodology. Reviews of ISO 9241-9 are given by Douglas, Kirkpatrick, and MacKenzie (1999); Douglas and Mithal (1997, chap. 8); and Smith (1996, chap. 3). At least six pointing-device studies have been published wherein the methodology conforms to ISO 9241-9 (Douglas et al., 1999; MacKenzie & Jusoh, 2001; MacKenzie, Kauppinen, & Silfverberg, 2001; MacKenzie & Oniszczak, 1998; Oh & Stuerzlinger, 2002; Silfverberg, MacKenzie, & Kauppinen, 2001).

Another active area of Fitts' Law research is in extending its application to a different genre of tasks. Until recently, Fitts' Law was applied only in *time-minimizing tasks* (or *rapid aimed movements*). *Space-minimizing tasks* are distinctly different. In a space-minimizing task, users move a tracking symbol such as a cursor while trying to minimize the deviation of the symbol from a predefined path. Accot and Zhai (1997) effectively showed how to model this behavior using a variation of Fitts' Law aptly named the *Steering law*. Examples include navigating

hierarchical menus, manipulating an elevator on a scrollbar, or tracing the outline of a shape. Besides the clear need for a prediction model for this genre of tasks, this work is important because it seeks to unify distinctly different motor behaviors within a single theoretical framework. Two related follow-up studies have also appeared (Accot & Zhai, 1999; Accot & Zhai, 2001).

Guiard's work is a continuing source of inspiration for designers of systems for interactive 3D graphics. These systems are generally high-end workstations with graphics accelerators and high-resolution displays. They often support input via graphics tablets and a stylus. The interface is complex, supporting a vast array of functionality. And, so, efforts to improve the interface through two-handed interaction are well invested. Several sources of this work are hereby cited (e.g., Balakrishnan & Hinckley, 2000; Balakrishnan & Kurtenbach, 1999; Fitzmaurice, Balakrishnan, Kurtenbach, & Buxton, 1999; Hinckley, Pausch, Proffitt, & Kassell, 1998; Hinckley, Pausch, Proffitt, Patten, & Kassell, 1997; Kurtenbach, Fitzmaurice, Baudel, & Buxton, 1997; Zhai, Smith, & Selker, 1997a; Zhai, Smith, & Selker, 1997b).

For readings on other predictive models developed with an eye to human-computer interfaces, the reader is directed to several sources. Examples include the Goals, Operators, Methods, and Selection rules (GOMS) model by Card, Moran, and Newell (Card et al., 1983; John, 1995; John & Kieras, 1996; Kieras, Wood, & Meyer, 1997), or the programmable user model (PUM) by Young, Green, and Simon (1989).

For other descriptive models on input devices, including movement characteristics, properties sensed, stimulus-response mappings, and interaction techniques, the reader is directed to several primary sources (Buxton, 1983; Card, Mackinlay, & Robertson, 1990; Card, Mackinlay, & Robertson, 1991; Foley, Wallace, & Chan, 1984; Jacob, Sibert, McFarlane, & Mullen, 1994; Mackinlay, Card, & Robertson, 1990; Ren & Moriya, 2000).